

Politechnika Gdańska
Wydział Elektroniki, Telekomunikacji i Informatyki

Sprawozdanie

Temat: Symulator Programowy Mikroprocesora 8086

Adam Błażejowski 198344
Mateusz Kuczerowski 197900
Data: 01.06.2026

1 Sformułowanie zadania i przyjęte założenia

Celem zadania było zaprojektowanie i zaimplementowanie aplikacji stanowiącej model programowego symulatora mikroprocesora w standardzie PC. Program został rozszerzony o obsługę wybranych przerwań systemowych (BIOS/DOS), łącząc funkcje wykonawcze z modułem dydaktycznym.

1.1 Założenia szczegółowe programu

- **Architektura procesora:** Zaimplementowano cztery 16-bitowe rejestry ogólnego przeznaczenia: AX, BX, CX, DX. Każdy z nich może być adresowany jako para rejestrów 8-bitowych (np. AH dla starszej części i AL dla młodszej części rejestru AX).
- **Lista rozkazów:** Symulator obsługuje podstawowe operacje: MOV (przesyłanie danych), ADD (dodawanie), SUB (odejmowanie), PUSH (odkładanie na stos) oraz POP (pobieranie ze stosu).
- **Tryby adresowania:** Program realizuje dwa tryby adresowania: rejestrowy (operacje między rejestrami) oraz natychmiastowy (przypisywanie stałych wartości liczbowych, obsługiwane w formacie dziesiętnym, szesnastkowym i binarnym).
- **Tryby wykonania:** Umożliwiono uruchamianie kodu w trybie całościowym oraz krokowym. Tryb krokowy pozwala na śledzenie przebiegu programu, wykorzystując podświetlanie aktualnie wykonywanej linii w edytorze kodu.
- **Zarządzanie plikami:** Zaimplementowano funkcjonalność zapisu napisanego programu do pliku tekstowego oraz jego ponownego wczytania.
- **Obsługa przerwań (Zadanie B):** Do listy komend dodano instrukcję INT. Program symuluje różnorodne funkcje przerwań (m.in. obsługa konsoli, pobieranie czasu RTC). Na czas wykonywania przerwania, stan rejestrów jest zabezpieczany przy pomocy stosu wewnętrznego.
- **Moduł dydaktyczny:** Aplikacja posiada wbudowaną dokumentację (osobne zakładki dla instrukcji assemblera oraz przerwań), która szczegółowo wyjaśnia sposób działania poszczególnych komend.

2 Opis przyjętych rozwiązań programowych

Aplikacja została wykonana w środowisku .NET ze wsparciem biblioteki Windows Forms przy użyciu języka C#. Interfejs graficzny został podzielony na zakładki, separując edytor kodu od modułu dokumentacji. Rejestry procesora reprezentowane są za pomocą zmiennych typu `ushort`, a stos sprzętowy przy użyciu struktury `Stack<ushort>`.

2.1 Parsowanie linii programu i tryb krokowy

Kluczowym elementem symulatora jest metoda analizująca składnię. Kod źródłowy jest dzielony na linie, a każda instrukcja rozbijana na leksemy. Poniższy fragment kodu (Listing 1) prezentuje logikę pracy krokowej, która dba o bezpieczeństwo wykonania i wizualizację postępu:

Listing 1: Fragment metody obsługującej tryb pracy krokowej wraz z podświetlaniem linii.

```
private void button_step_mode_Click(object sender, EventArgs e) {
    if (isProgramTerminated) {
        MessageBox.Show("Program został zakończony. Zresetuj układ.");
        return;
    }
    if (curr_line == 0) {
        isProgramTerminated = false;
        program_lines_step_work = program_display.Text
            .Split(new[] { '\r', '\n' }, StringSplitOptions.
                RemoveEmptyEntries)
            .Select(line => line.Trim()).ToArray();
    }
    if (curr_line >= program_lines_step_work.Length) {
        curr_line = 0; return;
    }
    textBox_curr_line.Text = (curr_line + 1).ToString();
    highlight_current_line(curr_line);
    instructionExec(program_lines_step_work[curr_line]);
    refreshAllReg();
    curr_line++;
}
```

2.2 Implementacja operacji arytmetycznych i przesyłania danych

Aby obsłużyć zarówno 16-bitowe, jak i 8-bitowe odwołania do rejestrów, stworzono uniwersalne metody modyfikujące referencje do rejestrów z uwzględnieniem masek bitowych.

Listing 2: Operacja dodawania z uwzględnieniem modyfikatorów wielkości rejestru (X, H, L).

```
private void ADD(ref ushort reg, char flag, ushort value) {
    if (flag == 'X') {
        reg = (ushort)(reg + value);
    } else if (flag == 'H') {
        reg = (ushort)((reg & 0x00FF) | (reg + (value << 8)));
    } else if (flag == 'L') {
        reg = (ushort)((reg & 0xFF00) | ((reg + value) & 0x00FF));
    }
}
```

2.3 Symulacja przerwań systemowych

Przerwania wywoływane instrukcją INT analizują zawartość odpowiednich rejestrów (najczęściej AH i AL), by zidentyfikować pożądaną podfunkcję.

Listing 3: Fragment instrukcji warunkowej obsługującej podfunkcję przerwania INT 21H.

```
switch (intNum) {
    case 0x21:
        switch (ah) {
            case 0x02: // Wypisz pojedynczy znak z rejestru DL
                char charToPrint = (char)(D & 0xFF);
                outputConsoleTextBox.AppendText(charToPrint.ToString());
                break;
            case 0x2A: // Pobierz datę systemowa
```

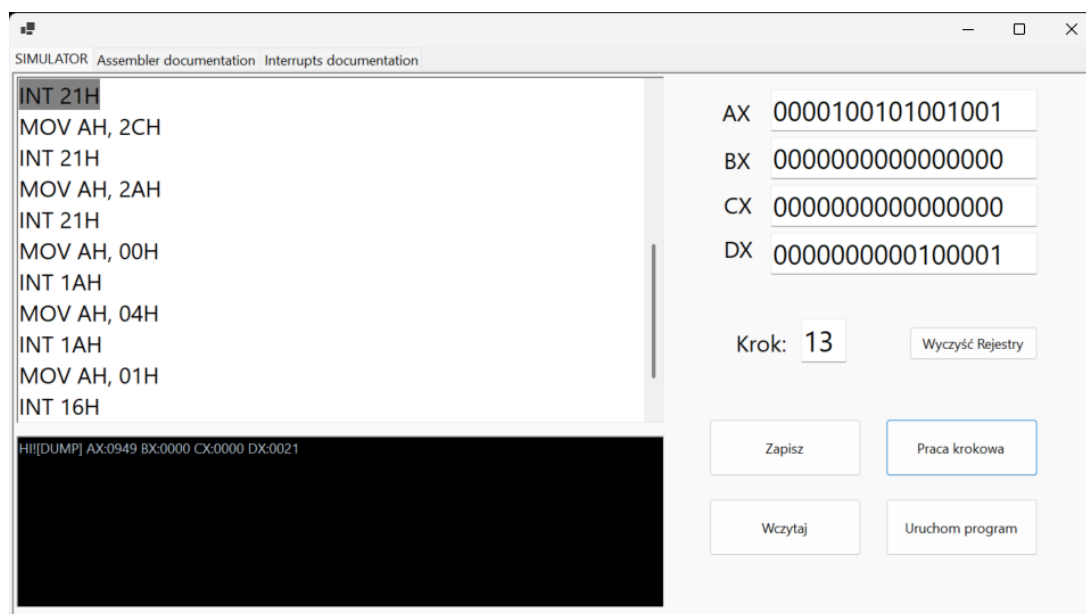
```

        DateTime date = DateTime.Now;
        C = (ushort)date.Year;
        D = (ushort)((date.Month << 8) | date.Day);
        break;
    case 0x4C: // Zakoncz program
        isProgramTerminated = true;
        outputConsoleTextBox.AppendText("\n---Program
            zakonczony---\n");
        break;
    }
    break;
}

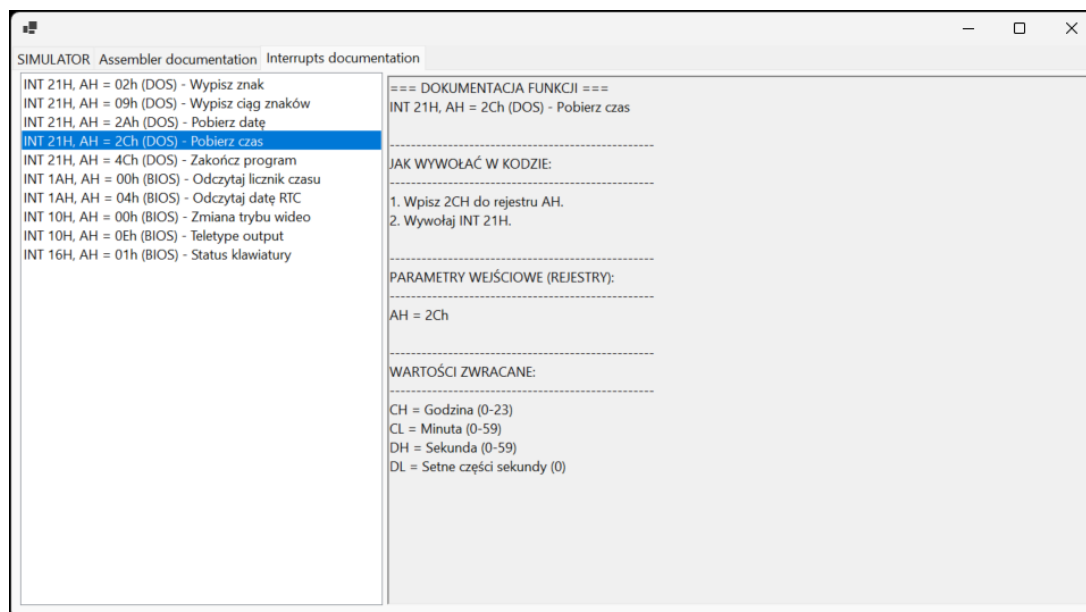
```

3 Prezentacja wyników i funkcjonalności

Zgodnie z wymaganiami, aplikacja posiada przejrzysty interfejs graficzny. Poniżej zaprezentowano rozmieszczenie głównych elementów systemu.



Rysunek 1: Główny widok symulatora prezentujący edytor kodu, wskaźniki rejestrów oraz okno konsoli wyjściowej.



Rysunek 2: Moduł dydaktyczny z dokumentacją przerwań, zawierający opis składni, parametrów wejściowych oraz wartości zwracanych.

3.1 Zalety aplikacji

- **Rozbudowany aspekt dydaktyczny:** Aplikacja integruje w sobie środowisko uruchomieniowe oraz pełną dokumentację komend i przerwań, co ułatwia naukę podstaw programowania w asemblerze.
- **Wizualizacja pracy krokowej:** Możliwość wykonywania instrukcji linia po linii z jednoczesnym podglądem zmieniających się stanów rejestrów drastycznie upraszcza proces debugowania.
- **Bezpieczeństwo i obsługa błędów:** Zastosowano bloki try-catch przy parsowaniu wartości liczbowych (ochrona przed przepełnieniem). Wywołanie POP na pustym stosie generuje czytelny komunikat błędu zamiast awarii programu.

3.2 Wady i ograniczenia napisanego programu

- **Monolityczna architektura:** Logika symulacji mikroprocesora jest silnie sprzężona z kodem interfejsu użytkownika (plik `Form1.cs`). Brak wyraźnego podziału na warstwy (np. MVC) utrudnia ewentualną rozbudowę czy wprowadzanie testów jednostkowych.
- **Ograniczona lista instrukcji:** Zaimplementowano jedynie ułamek rzeczywistej listy rozkazów procesora 8086. Brakuje instrukcji skoków warunkowych i bezwarunkowych (JMP, JZ, itp.), co uniemożliwia tworzenie pętli.
- **Brak pamięci operacyjnej:** Architektura nie uwzględnia w pełni segmentacji pamięci (CS, DS, SS, ES) ani swobodnego dostępu do komórek pamięci RAM.

4 Podsumowanie

Zaprojektowana i zaimplementowana aplikacja w środowisku Windows Forms spełnia główne założenia sformułowane w zadaniach 5 oraz B. Symulator z powodzeniem odwzorowuje działanie rejestrów procesora klasy PC, realizując wybrane instrukcje assemblerowe oraz symulując elementarne przerwania sprzętowe i systemowe.

Dzięki zintegrowaniu modułu ewaluacyjnego z modułem dydaktycznym, aplikacja stanowi użyteczne narzędzie edukacyjne, demonstrujące niskopoziomowe mechanizmy działania komputera. Mimo uproszczeń architektonicznych, program tworzy solidną podstawę do dalszego rozwoju o zaawansowane mechanizmy, takie jak pełna obsługa pamięci operacyjnej czy sterowanie przepływem.